

UNINASSAU

Curso: Sistemas de Informação

nassauTickets

Sistema de Controle de Atendimento para um Laboratório de Análises
Clínicas

Olinda

2026

UNINASSAU

nassauTickets

Sistema de Controle de Atendimento para um Laboratório de Análises Clínicas

Relatório acadêmico apresentado à UNINASSAU como registro do desenvolvimento e da documentação do projeto nassauTickets. Curso, disciplina, docente, turma e natureza específica da avaliação: INFORMAÇÃO NÃO FORNECIDA.

Integrantes e papéis

Nome	Matrícula	Papel
Davi Henrique	24009979	Scrum Master
Igor Santos	01815246	Documentador
Lucas José	01815708	Desenvolvedor e Documentador
Pedro Vieira	01679503	Testador

Olinda
2026

Sumário

1. Resumo
2. Síntese do relatório
3. Introdução
4. Contextualização do projeto
5. Problema e justificativa
6. Objetivos
 - 6.1 Objetivo geral
 - 6.2 Objetivos específicos
7. Desenvolvimento do projeto
 - 7.1 Descrição do sistema
 - 7.2 Tecnologias utilizadas
 - 7.3 Arquitetura e visão geral do sistema
 - 7.4 Regras de negócio e funcionamento
 - 7.5 Principais telas e funcionalidades
 - 7.6 Acessibilidade e identidade visual
 - 7.7 Estrutura do projeto
 - 7.8 Instalação e configuração
 - 7.9 Controle de versão e branches
8. Participação dos integrantes
9. Resultados e estado atual
10. Considerações finais
11. Referências

Resumo

Este relatório apresenta o projeto nassauTickets, um sistema de controle de atendimento para um laboratório de análises clínicas da UNINASSAU. Na versão documentada, o cliente emite uma senha em um totem, acompanha as chamadas em um painel e o atendente utiliza um console de guichê para chamar, iniciar e encerrar atendimentos. O frontend foi implementado com React, Vite, React Router e CSS puro. A fase atual utiliza armazenamento no navegador por meio de `localStorage` e a Web Locks API para reduzir conflitos entre guichês; a documentação registra um backend com Node.js, Express e MySQL como etapa futura. O relatório descreve os requisitos, as regras de negócio, a estrutura do código, o fluxo de uso, as instruções de instalação e o estado atual, mantendo explícitos os dados institucionais que não foram fornecidos no material analisado.

Palavras-chave: sistema de atendimento; gerenciamento de filas; React; acessibilidade; laboratório de análises clínicas.

Síntese do relatório

Este relatório descreve o sistema nassauTickets, seus requisitos, sua organização técnica, suas telas, as regras de negócio e o estado atual da implementação. A versão analisada corresponde a um frontend React executado localmente, com armazenamento da fila no navegador e previsão de evolução para uma arquitetura com backend.

Termos-chave: sistema de atendimento; gerenciamento de filas; React; acessibilidade; laboratório de análises clínicas.

Introdução

O nassauTickets é um projeto de software voltado ao controle de atendimento em um laboratório de análises clínicas. A necessidade registrada no material do projeto é organizar a emissão e a chamada de senhas, separando atendimentos prioritários, gerais e de retirada de exames. O sistema procura tornar visível o andamento da fila para o cliente e oferecer ao atendente um fluxo controlado para chamar, iniciar e encerrar cada atendimento.

A versão analisada apresenta três pontos de interação: o totem, no qual a senha é emitida sem login; o painel, que mostra as chamadas recentes e o guichê correspondente; e a tela do atendente, que exige login demonstrativo e concentra as operações do guichê. O presente relatório transforma a documentação técnica em uma descrição acadêmica do sistema, sem atribuir ao projeto tecnologias ou funcionalidades que não estejam comprovadas nos arquivos disponíveis.

Contextualização do projeto

O nome do projeto é nassauTickets e sua descrição no README o caracteriza como um “Sistema de Controle de Atendimento para um Laboratório de Análises Clínicas (UNINASSAU)”. O fluxo começa quando o cliente escolhe um tipo de atendimento no totem. Os tipos existentes são SP, para atendimento prioritário; SG, para atendimento geral; e SE, para retirada de exames. Após a emissão, a senha aguarda na fila, pode ser chamada por um guichê e segue seu ciclo de atendimento.

O projeto foi desenvolvido para uma situação em que diferentes guichês precisam compartilhar uma fila. A versão atual não utiliza um backend operacional: o serviço `frontend/src/services/fila.js` mantém o estado no navegador. O README informa que a substituição desse serviço por chamadas REST a um backend pertence à fase 2.

Problema e justificativa

O problema abordado é a necessidade de organizar a espera e o atendimento de clientes com diferentes prioridades. Sem uma fila controlada, a chamada pode não respeitar a alternância prevista, o painel pode não refletir o guichê responsável e dois atendentes podem tentar receber a mesma senha. O projeto trata esses pontos por meio de regras de seleção, estados de senha, registro local das chamadas e sincronização entre abas do navegador.

A relevância do desenvolvimento está na centralização do fluxo de atendimento em uma interface simples para cada participante. O cliente recebe uma identificação e acompanha a chamada; o atendente dispõe de ações explícitas; e o painel apresenta somente as últimas cinco chamadas, sem revelar a próxima senha da fila.

Objetivos

Objetivo geral

Desenvolver e documentar um sistema de controle de atendimento capaz de emitir, organizar, chamar e acompanhar senhas de um laboratório de análises clínicas, respeitando as prioridades e o ciclo de atendimento definidos no projeto.

Objetivos específicos

- Disponibilizar a emissão de senhas SP, SG e SE sem login.
- Aplicar a alternância de chamadas definida nas regras de negócio.
- Permitir ao atendente chamar a próxima senha pelo guichê selecionado.
- Registrar chamadas repetidas, início, encerramento e não comparecimento.
- Exibir no painel a chamada atual e até quatro chamadas anteriores.
- Manter foco visível, contraste alto, navegação por teclado, áudio de chamada e *aria-live* no painel.
- Documentar a instalação e a execução do frontend.

Desenvolvimento do projeto

Descrição do sistema

O sistema é composto por uma aplicação frontend React com rotas para as telas `/`, `/painel` e `/atendente`. A rota inicial representa o totem; `/painel` representa o painel de chamadas; e `/atendente` representa o login e o console do guichê. O cabeçalho apresenta a marca, a navegação principal e uma opção de demonstração que ignora o horário de expediente.

O armazenamento atual é feito em `localStorage`, usando a chave `nassauTickets:v1`. A aplicação também dispara eventos locais para atualizar outras telas abertas no mesmo navegador e realiza uma atualização periódica a cada três segundos.

Tecnologias utilizadas

Tecnologia	Uso	Relação com o projeto
React 19	Interface	Construção dos componentes e das páginas da aplicação.
Vite 6	Desenvolvimento	Servidor de desenvolvimento e processo de build do frontend.
React Router	Navegação	Definição das rotas do totem, painel e atendente.
JavaScript	Lógica	Implementação do serviço de fila e dos componentes.
CSS puro	Apresentação	Cores, tipografia, responsividade, foco e animações.
localStorage	Persistência local	Armazenamento da fila no navegador na fase atual.
Web Locks API	Concorrência	Bloqueio da transação local para reduzir a emissão duplicada entre guichês.
Node.js, Express e MySQL	Fase futura	Tecnologias mencionadas no README para a fase 2; não há backend implementado nos arquivos analisados.

Arquitetura e visão geral do sistema

Não há, nos arquivos analisados, uma declaração formal de uma arquitetura como MVC ou outra classificação arquitetural. É possível, porém, observar uma organização por páginas e serviço: `src/pages` concentra as telas; `src/services/fila.js` concentra o estado e as regras da fila; `App.jsx` configura as rotas e o layout principal; e `styles.css` concentra a apresentação.

A comunicação entre as telas da fase 1 ocorre no próprio navegador. A função `transacao` lê o estado, aplica uma alteração, grava o resultado e dispara o evento `nassau:mudou`. Quando disponível, a Web Locks API envolve a operação com o bloqueio `nassauTickets`. Essa solução é local e não substitui a arquitetura de backend descrita como objetivo da fase 2.

Regras de negócio e funcionamento

A seleção da próxima senha segue as regras RN02 a RN04. Depois de uma SP, o sistema procura SE ou SG e, em seguida, SP. Depois de SE, procura SP e depois SG ou SE; em situação inicial, procura SP, SE e SG. Se uma das filas estiver vazia, a próxima alternativa disponível é utilizada. Um guichê não pode chamar nova senha enquanto possui um atendimento ativo.

A senha possui estados EMITIDA, AGUARDANDO, CHAMADA, CHAMADA_NOVAMENTE, EM_ATENDIMENTO, ATENDIDA e NÃO_COMPARECEU. A primeira chamada registra o guichê, o atendente, o horário e o estado CHAMADA. A ação “Chamar novamente” muda o estado para CHAMADA_NOVAMENTE e repete o anúncio com o aviso “Última chamada”. O atendente pode iniciar o atendimento, encerrá-lo ou registrar o não comparecimento.

A numeração segue o formato observado no código YYMMDD-TIPONNN, como em 260101-SP001; o README descreve o formato de maneira abreviada como YYMMDD-PPSQ. A sequência é mantida separadamente por

tipo e reiniciada quando o dia muda. Essa diferença de representação entre o README e a montagem efetiva da string deve ser revisada pelo grupo antes de uma entrega final.

O expediente é definido entre 7h e 17h. Fora dele, o sistema bloqueia a emissão e a chamada, salvo quando o modo de demonstração está habilitado. Senhas aguardando são marcadas como descartadas ao final do expediente ou quando pertencem a outro dia.

Principais telas e funcionalidades

Totem

A tela apresenta três botões: atendimento prioritário, atendimento geral e retirada de exames. Cada botão exibe a quantidade aguardando. Ao emitir, o sistema mostra a senha recebida e informa que o cliente deve acompanhar o painel. A emissão não requer login.

Painel

O painel apresenta a chamada atual, seu número e o guichê. Abaixo, quando existem registros, apresenta as chamadas anteriores. A função `ultimasChamadas` ordena as chamadas pelo horário mais recente e limita o resultado a cinco. Quando a leitura do estado falha, a interface informa que está exibindo a última chamada conhecida.

Atendente

O login demonstrativo utiliza os dados documentados no README e permite escolher os guichês de 1 a 4. Depois de entrar, o atendente visualiza as quantidades aguardando por tipo e as ações disponíveis. Os botões são habilitados de acordo com o estado atual da senha, evitando iniciar ou encerrar operações incompatíveis com o fluxo.

Acessibilidade e identidade visual

O projeto define contraste alto, foco visível para navegação por teclado, link para saltar ao conteúdo, `aria-live` no painel, áudio das chamadas e respeito à preferência `prefers-reduced-motion`. A identidade visual usa fundo verde clínico, cartões verde profundo, verde neon, ciano hospitalar, rosa neon e menta clara. Os títulos e números usam Sora; o texto geral usa Atkinson Hyperlegible, escolhida no projeto por sua legibilidade.

Estrutura do projeto

Caminho	Finalidade observável
<code>frontend/src/App.jsx</code>	Componente principal, layout, navegação e modo de demonstração.
<code>frontend/src/pages/</code>	Componentes das telas Totem, Painel e Atendente.
<code>frontend/src/services/fila.js</code>	Estado local da fila, emissão, seleção, transições e áudio.
<code>frontend/styles.css</code>	Estilos globais, tema, responsividade e acessibilidade visual.
<code>frontend/public/logo.svg</code>	Marca utilizada pelo frontend.
<code>docs/requirements/</code>	Requisitos funcionais, não funcionais, regras de negócio e casos de uso.
<code>docs/models/uml/</code>	Máquina de estados documentada em Mermaid.

Caminho	Finalidade observável
docs/mer/	MER inicial, incluindo entidades previstas para a fase com backend.
backend/	Diretório presente apenas com .gitkeep na versão analisada.

Instalação e configuração

Os pré-requisitos comprováveis são Node.js e npm, pois o projeto contém package.json, package-lock.json e scripts do npm. No README, a execução indicada é:

```
cd frontend
npm install
npm run dev
```

Depois, deve-se acessar <http://localhost:5173>. Para testar vários guichês, o README orienta abrir / atendente em abas diferentes. O arquivo abrir-nassauTickets.bat oferece uma alternativa para Windows: instala dependências se necessário, abre o endereço local e inicia npm run dev.

A configuração de demonstração possui uma caixa chamada “Ignorar horário de expediente”. O README registra credenciais demonstrativas para a tela de atendente; por segurança, este relatório não as reproduz. Em uma implementação real, a autenticação deve ser substituída pelo backend, conforme indicado no próprio README e nos requisitos não funcionais.

Controle de versão e branches

O README informa duas branches: main, descrita como versão estável que recebe merge da dev, e dev, utilizada no desenvolvimento diário. Não foram encontrados, no material disponibilizado, dados sobre URL de repositório, commits específicos, pull requests ou responsáveis por merges. Portanto, essas informações permanecem como INFORMAÇÃO NÃO FORNECIDA.

Participação dos integrantes

O README identifica quatro integrantes e atribui os papéis abaixo. Como o material não detalha tarefas individuais, as contribuições são descritas de forma compatível com os papéis informados, sem afirmar atividades específicas não registradas.

Integrante	Papel informado	Contribuição individual
Davi Henrique	Scrum Master	Organização do fluxo de trabalho e acompanhamento das etapas do projeto.
Igor Santos	Documentador	Levantamento, organização e redação da documentação do projeto.
Lucas José	Desenvolvedor e Documentador	Criação, formatação e organização da documentação técnica do projeto, com apoio pontual na manutenção do frontend.
Pedro Vieira	Testador	Verificação do fluxo de emissão, chamadas e estados da fila.

Resultados e estado atual

O material analisado demonstra uma fase 1 de frontend funcional em navegador, com totem, painel, tela de atendente, emissão de três tipos de senha, alternância de chamadas, controle de estados, áudio, armazenamento local e sincronização entre abas. A estrutura de documentação também registra requisitos funcionais e não funcionais, regras de negócio, casos de uso, máquina de estados, MER inicial e identidade visual.

O backend não está implementado na pasta analisada: backend contém apenas .gitkeep. Relatórios de gestor, autenticação real, armazenamento MySQL, API REST e auditoria persistente são apresentados como fase 2 ou requisitos futuros. Não foram encontrados testes automatizados nem resultados de testes documentados; portanto, este relatório não afirma resultados de validação que não estejam disponíveis.

Entre as limitações observáveis estão a persistência restrita ao navegador, o login demonstrativo no frontend e a dependência de APIs do navegador para bloqueio e fala. Também foi identificada a divergência de representação da numeração entre o README e o código, mencionada anteriormente, que requer decisão do grupo.

Considerações finais

O nassauTickets apresenta uma solução de frontend para organizar a emissão e o atendimento de senhas em um laboratório de análises clínicas. O projeto possui uma separação clara entre telas e serviço de fila, regras explícitas de prioridade, estados documentados e uma identidade visual voltada à legibilidade e à acessibilidade.

Na versão analisada, o objetivo principal é atendido por uma aplicação executável com Vite, mas a solução ainda está delimitada à fase local. A evolução prevista no próprio material é a criação do backend com Node.js, Express e MySQL, juntamente com autenticação, relatórios e persistência centralizada. Antes de uma entrega final, recomenda-se revisar a divergência no formato da senha, completar os dados institucionais ausentes, registrar evidências de testes e documentar o repositório, sem alterar a distinção entre funcionalidades presentes e planejadas.

Referências

GRUPO NASSAUTICKETS. **README.md do projeto nassauTickets**. Material fornecido no arquivo nassauTickets-corrigido.zip, 2026.

GRUPO NASSAUTICKETS. **Documentação de requisitos, regras de negócio, casos de uso, MER, identidade visual e máquina de estados**. Material fornecido no arquivo nassauTickets-corrigido.zip, 2026.

GRUPO NASSAUTICKETS. **Código-fonte do frontend**. Material fornecido no arquivo nassauTickets-corrigido.zip, 2026.

UNINASSAU. **Atividade - Sistema para controle de atendimento**. Material de referência disponibilizado pelo professor. Dados editoriais e data: [INFORMAÇÃO NÃO FORNECIDA].

UNINASSAU. **Sistema para controle de atendimento**. Material de referência disponibilizado pelo professor. Dados editoriais e data: [INFORMAÇÃO NÃO FORNECIDA].